Informe de práctica: Modelos Transformers en procesamiento de lenguaje natural

Juan Nicolás Romero Estupiñan

Universidad De Cundinamarca

Inteligencia Artificial - Conceptos

Paul Alexander Diaz Montaña

Noviembre 2026

Informe de práctica: Modelos Transformers en procesamiento de lenguaje natural

Los modelos Transformers son la base de la mayoría de los sistemas actuales de procesamiento de lenguaje natural (NLP). Propuestos por Vaswani et al. (2017), sustituyeron las capas recurrentes y convolucionales por un mecanismo de atención, lo que permite procesar todas las posiciones de una secuencia en paralelo y entrenar con mucho menos tiempo.

Este informe resume los conceptos clave de la arquitectura, describe una práctica realizada en Google Colab con la librería Transformers de Hugging Face, presenta los resultados obtenidos y los analiza a la luz de las lecturas asignadas: el artículo original de Vaswani et al. (2017) y la explicación visual de Doshi (2020). Se acompaña de un mapa mental que organiza los conceptos principales.

Conceptos clave

Fundamentos de los Transformers

Los modelos de transducción de secuencias, como los usados en traducción automática, dominaban con redes recurrentes (RNN, LSTM, GRU). Estas calculan un estado oculto por posición a partir del estado anterior, lo que impide paralelizar el cálculo dentro de cada ejemplo de entrenamiento (Vaswani et al., 2017). Las redes convolucionales sí paralelizan, pero el número de operaciones necesarias para relacionar dos posiciones crece con la distancia entre ellas.

El Transformer propone una arquitectura basada solo en atención, sin recurrencia ni convoluciones. Relaciona dos posiciones cualesquiera con un número constante de operaciones, y en su versión base se entrenó en 12 horas con 8 GPU P100 (Vaswani et al., 2017). Doshi (2020) resume la ventaja: el modelo procesa todas las palabras en paralelo y la distancia entre ellas no afecta su capacidad de modelar dependencias.

Tabla 1

Comparación de tipos de capa según Vaswani et al. (2017)

Tipo de capa	Complejidad por capa	Operaciones secuenciales	Longitud máxima de camino
Auto-atención	$O(n^2 \cdot d)$	O(1)	O(1)
Recurrente	$O(n \cdot d^2)$	O(n)	O(n)
Convolucional	$O(k \cdot n \cdot d^2)$	O(1)	$O(\log_k n)$

Nota. n = longitud de la secuencia, d = dimensión de la representación, k = tamaño del kernel. Adaptado de Vaswani et al. (2017, Tabla 1).

Mecanismo de atención

Una función de atención asigna una consulta (*query*, Q) y un conjunto de pares clave-valor (*key*, K; *value*, V) a una salida: una suma ponderada de los valores, donde cada peso resulta de comparar la consulta con la clave correspondiente. El Transformer usa la atención de producto punto escalado (Vaswani et al., 2017):

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})\, V$$

El factor $1/\sqrt{d_k}$ evita que, con dimensiones grandes, los productos punto crezcan tanto que la función softmax quede en zonas de gradientes muy pequeños. En lugar de una sola función de atención, se usa atención multi-cabeza: las proyecciones lineales de Q, K y V se repiten h = 8 veces en paralelo, con $d_k = d_v = 64$, y los resultados se concatenan y se proyectan de nuevo. Así el modelo atiende a información de distintos subespacios de representación a la vez.

El modelo aplica la atención de tres formas (Vaswani et al., 2017):

- **Auto-atención del encoder:** cada posición atiende a todas las posiciones de la capa anterior.
- **Auto-atención enmascarada del decoder:** cada posición solo atiende a las anteriores y a sí misma, para conservar la propiedad autorregresiva.

- **Atención codificador-decodificador:** las consultas vienen del decoder y las claves y valores, de la salida del encoder.

Doshi (2020) ilustra la idea con una frase en la que "it" se asocia con "cat" o con "milk" según el contexto: la atención permite relacionar cada palabra con todas las demás de la secuencia.

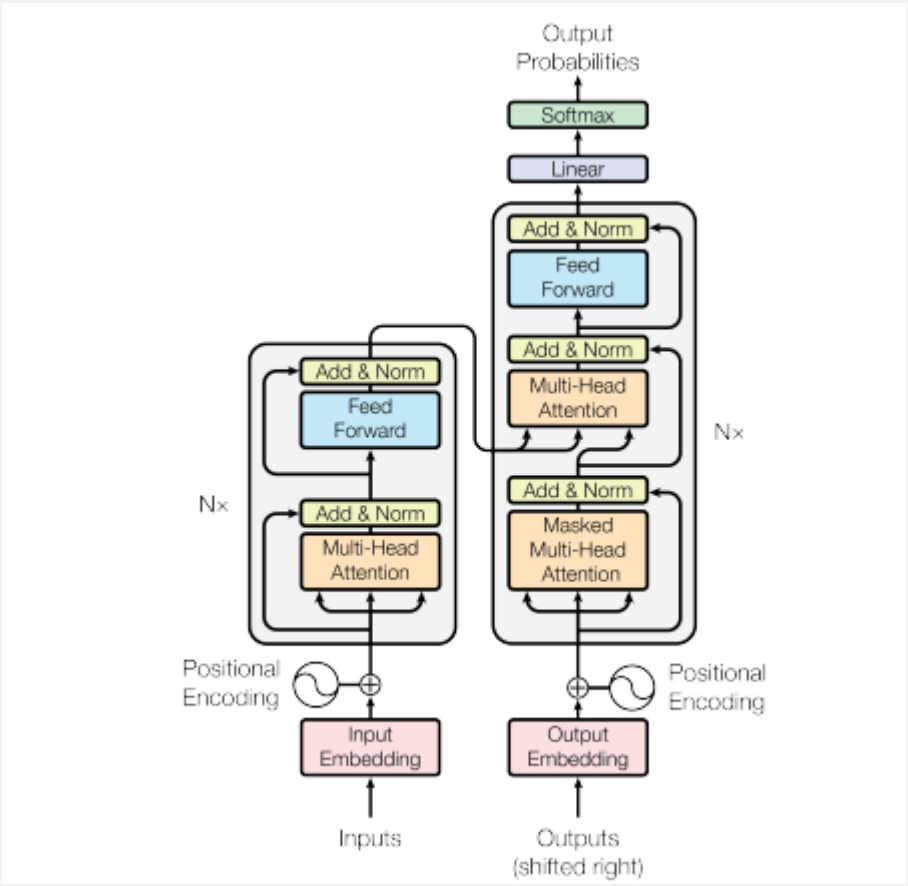
Estructura del Transformer

La arquitectura original sigue el esquema codificador-decodificador (Vaswani et al., 2017):

- **Encoder:** pila de N = 6 capas idénticas. Cada capa tiene dos subcapas: auto-atención multi-cabeza y una red *feed-forward* aplicada a cada posición, ambas con conexión residual y normalización de capa.
- **Decoder:** pila de N = 6 capas con una tercera subcapa de atención sobre la salida del encoder, y su auto-atención enmascarada.
- **Dimensiones:** $d_{model} = 512$ y $d_{ff} = 2048$ en el modelo base.
- **Embeddings y capa final:** embeddings aprendidos, con la misma matriz de pesos compartida con la capa lineal previa al softmax.
- **Codificación posicional:** como no hay recurrencia, se suman funciones seno y coseno de distintas frecuencias a los embeddings para inyectar el orden de las palabras.

Figura 1

Arquitectura del Transformer



Nota. Tomado de Vaswani et al. (2017, Figura 1).

En entrenamiento, Doshi (2020) explica que el decoder recibe la secuencia objetivo correcta (*teacher forcing*), lo que acelera el aprendizaje y permite generar todas las palabras en paralelo. En inferencia solo se dispone de la entrada: el decoder parte de un token de inicio y genera una palabra por iteración hasta predecir el token de fin.

Aplicaciones en NLP

Doshi (2020) menciona traducción automática, resumen de texto, respuesta a preguntas, reconocimiento de entidades nombradas y reconocimiento de voz. Un mismo bloque base sirve para varias tareas cambiando solo la cabeza final: una cabeza de clasificación para análisis de

sentimiento o una cabeza de modelo de lenguaje para generar texto. De la arquitectura original derivaron tres familias, que se usaron en la práctica (Tabla 2).

Tabla 2

Familias de modelos Transformer y ejemplos usados en la práctica

Familia	Componentes usados	Ejemplo	Tarea en la práctica
Solo encoder	Auto-atención bidireccional	BERT (Devlin et al., 2018)	Sentimiento y relleno de máscara
Encoder-decoder	Encoder, decoder y atención cruzada	Marian (Junczys-Dowmunt et al., 2018)	Traducción inglés a español
Solo decoder	Auto-atención enmascarada	GPT-2 (Radford et al., 2019)	Generación de texto

Nota. BERT usa contexto izquierdo y derecho en todas sus capas y se adapta a cada tarea con una sola capa de salida adicional (Devlin et al., 2018). GPT-2 es un modelo de lenguaje entrenado con WebText, un conjunto de millones de páginas web (Radford et al., 2019).

Descripción de la práctica

La práctica consistió en probar tres familias de Transformers con tareas de NLP y observar su mecanismo de atención. El código del cuaderno (*notebook*) fue diseñado por Claude, un asistente de inteligencia artificial de Anthropic (2026), a solicitud del estudiante, y se ejecutó en Google Colab.

Entorno. Google Colab sin GPU (la ejecución usó CPU), Python 3.13, PyTorch y la librería Transformers de Hugging Face, versión 5.22.1. La API `pipeline()` de esa librería ofrece una interfaz de alto nivel para inferencia: basta indicar una tarea, un modelo y una entrada (Hugging Face, s. f.).

Actividades. Se realizaron seis pasos en el mismo cuaderno:

1. **Análisis de sentimiento:** tres textos (uno en inglés y dos en español) clasificados de 1 a 5 estrellas con un BERT multilingüe.

2. **Relleno de máscara:** BERT predice la palabra oculta `[MASK]` en dos frases en inglés y se listan sus cinco opciones más probables.
3. **Traducción inglés a español:** tres oraciones traducidas con un modelo Marian. En esta versión de la librería la traducción se programó de forma manual, cargando el tokenizador y el modelo (`AutoModelForSeq2SeqLM`) y llamando a `generate()`.
4. **Generación de texto:** GPT-2 continúa un texto inicial con decodificación voraz (`do_sample=False`), 30 tokens nuevos y semilla 42.
5. **Visualización de la atención:** se extraen los pesos de atención de `bert-base-uncased` (`output_attentions=True`) para la frase "The law will never be perfect, but its application should be just", similar a la de las Figuras 4 y 5 de Vaswani et al. (2017). Se grafica la capa 5 (índice 4) y se identifica, en cada cabeza, a qué palabra atiende más el token "its".
6. **Tamaño de los modelos:** conteo de parámetros de cada modelo.

La Tabla 3 resume los modelos empleados.

Tabla 3

Modelos utilizados en la práctica

Actividad	Modelo (Hugging Face)	Familia
Sentimiento	nlptown/bert-base-multilingual-uncased-sentiment	Solo encoder
Relleno de máscara y atención	bert-base-uncased	Solo encoder
Traducción	Helsinki-NLP/opus-mt-en-es	Encoder-decoder
Generación	gpt2	Solo decoder

Resultados obtenidos

Todos los valores provienen de la ejecución en Google Colab (CPU, Transformers 5.22.1). Después de cada tabla se incluye un espacio para la figura con la captura de la celda correspondiente.

Análisis de sentimiento

El modelo clasificó los tres textos en 3,16 segundos (Tabla 4 y Figura 2).

Tabla 4

Resultados del análisis de sentimiento

Texto	Etiqueta	Confianza
I love how fast this model learns!	5 estrellas	0,84
Este curso me pareció aburrido y confuso.	2 estrellas	0,51
La práctica de Transformers estuvo excelente.	5 estrellas	0,48

Figura 2

Salida de la celda de análisis de sentimiento



Relleno de máscara

La Tabla 5 muestra las cinco predicciones más probables de BERT para cada frase (Figura 3).

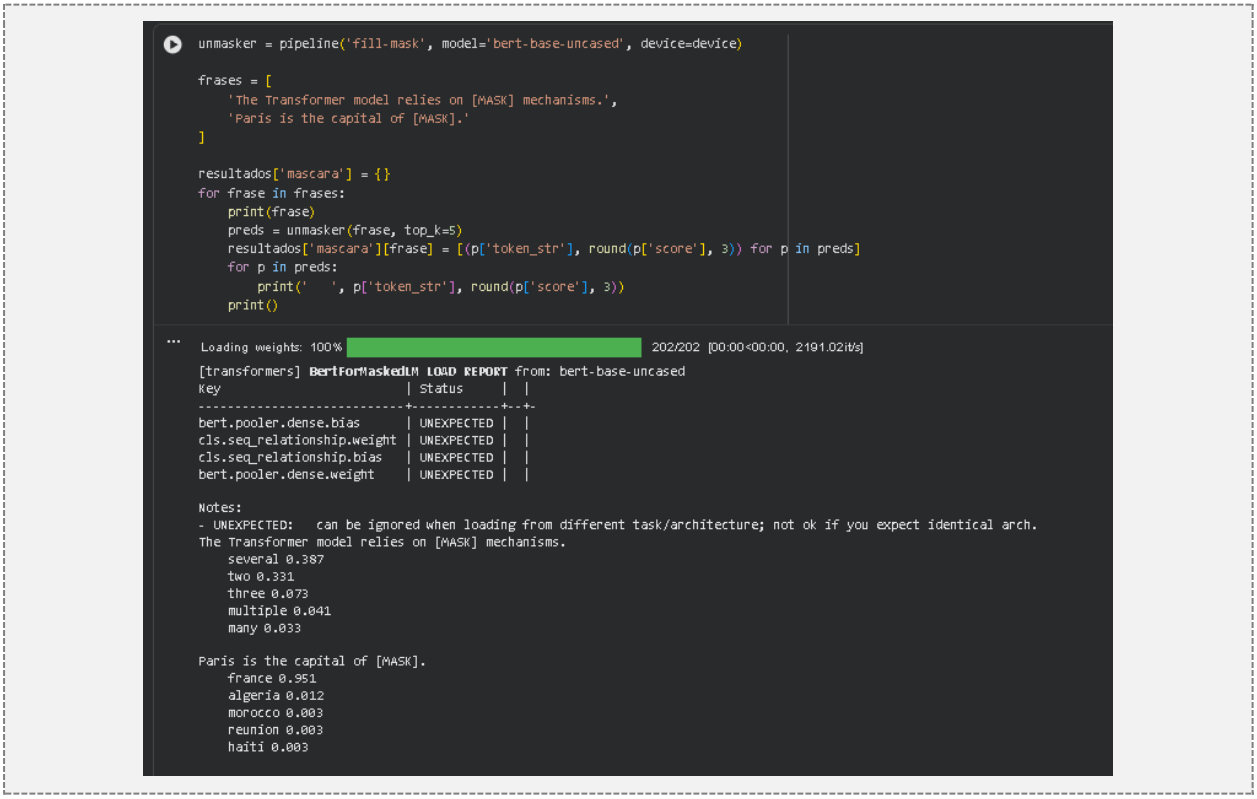
Tabla 5

Cinco predicciones más probables para la palabra enmascarada

Puesto	"The Transformer model relies on [MASK] mechanisms."	Prob.	"Paris is the capital of [MASK]."	Prob.
1	several	0,387	france	0,951
2	two	0,331	algeria	0,012
3	three	0,073	morocco	0,003
4	multiple	0,041	reunion	0,003
5	many	0,033	haiti	0,003

Figura 3

Salida de la celda de relleno de máscara



Traducción inglés a español

Las tres oraciones se tradujeron en 8,82 segundos (Tabla 6 y Figura 4).

Tabla 6

Traducciones generadas por el modelo

Inglés (entrada)	Español (salida)
Attention is all you need.	La atención es todo lo que necesitas.
The Transformer allows for significantly more parallelization.	El transformador permite una paralelización significativamente mayor.
The law will never be perfect, but its application should be just.	La ley nunca será perfecta, pero su aplicación debería ser justa.

Figura 4

Salida de la celda de traducción



```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Cargamos el tokenizador y el modelo de traducción directamente
model_name = 'Helsinki-NLP/opus-mt-en-es'
tok_traductor = AutoTokenizer.from_pretrained(model_name)
modelo_traductor = AutoModelForSeq2SeqLM.from_pretrained(model_name)
if device == 0:
    modelo_traductor = modelo_traductor.to('cuda')

oraciones = [
    'Attention is all you need.',
    'The Transformer allows for significantly more parallelization.',
    'The law will never be perfect, but its application should be just.'
]

t0 = time.time()

# Realizamos la traducción de forma manual
resultados['traduccion'] = []
for o in oraciones:
    inputs = tok_traductor(o, return_tensors='pt')
    if device == 0:
        inputs = {k: v.to('cuda') for k, v in inputs.items()}

    with torch.no_grad():
        translated_tokens = modelo_traductor.generate(**inputs)

    traduccion_texto = tok_traductor.batch_decode(translated_tokens, skip_special_tokens=True)[0]
    print('EN:', o)
    print('ES:', traduccion_texto, '\n')
    resultados['traduccion'].append((o, traduccion_texto))

# Guardamos una referencia compatible para el cálculo de parámetros posterior
traductor = type('DummyTraductor', (object,), {'model': modelo_traductor})

t = time.time() - t0
print('Tiempo (s):', round(t, 2))
```

```
/usr/local/lib/python3.13/dist-packages/transformers/models/marian/tokenization_marian.py:176: UserWarning: Recommended: pip install sacremoses.
  warnings.warn("Recommended: pip install sacremoses.")
Loading weights: 100% 258/258 [00:00<00:00, 19.98it/s]
EN: Attention is all you need.
ES: La atención es todo lo que necesitas.

EN: The Transformer allows for significantly more parallelization.
ES: El transformador permite una paralelización significativamente mayor.

EN: The law will never be perfect, but its application should be just.
ES: La ley nunca será perfecta, pero su aplicación debería ser justa.

Tiempo (s): 8.82
```

Generación de texto

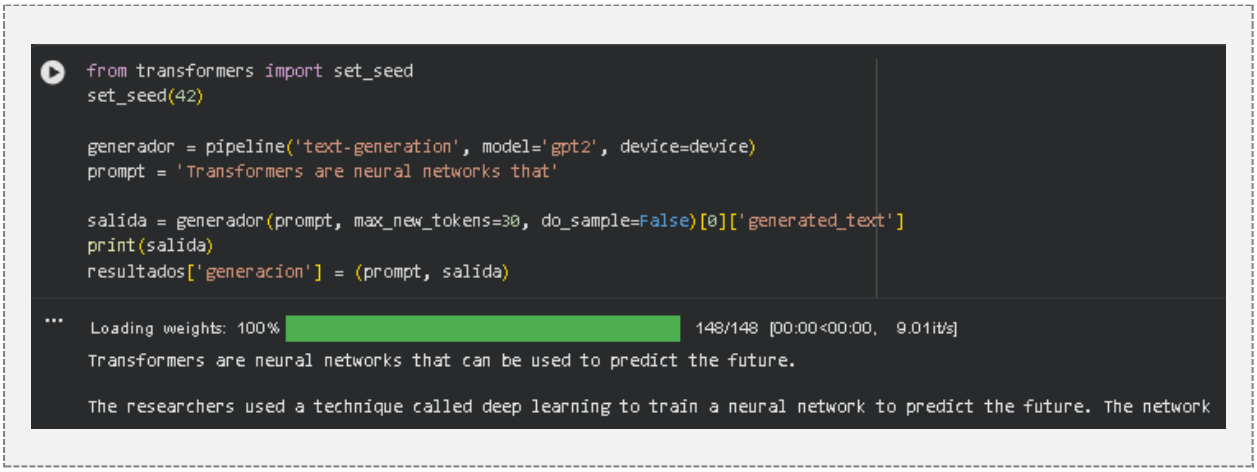
A partir del texto inicial "Transformers are neural networks that", GPT-2 generó la siguiente continuación (Figura 5):

```
Transformers are neural networks that can be used to predict the
future.

The researchers used a technique called deep learning to train a neural
network to predict the future. The network
```

Figura 5

Salida de la celda de generación de texto



Visualización de la atención

La frase se dividió en 15 tokens: [CLS], the, law, will, never, be, perfect, coma, but, its, application, should, be, just, [SEP]. El modelo tiene 12 capas con 12 cabezas cada una. La Tabla 7 indica, para cada cabeza de la capa 5, la palabra a la que más atiende el token "its"; la Figura 6 muestra los mapas de calor de esa capa.

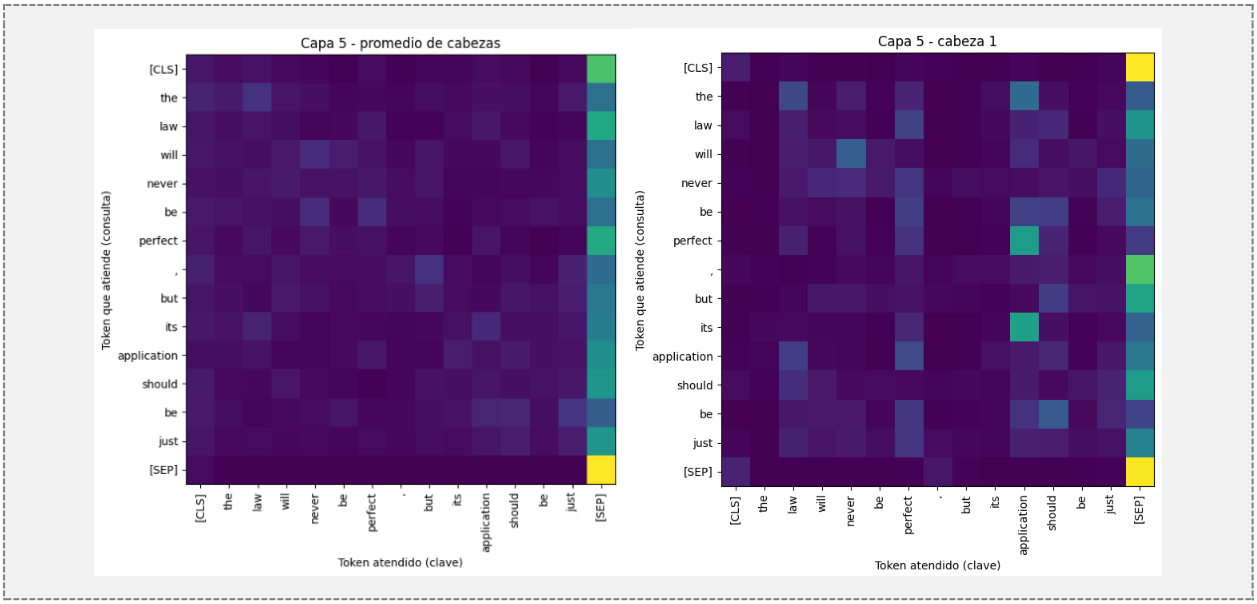
Tabla 7

Token con mayor peso de atención desde "its" en cada cabeza de la capa 5

Cabeza	Token más atendido	Peso
1	application	0,48
2	just	0,20
3	[SEP]	0,60
4	[SEP]	0,38
5	will	0,19
6	application	0,31
7	[SEP]	0,32
8	[SEP]	0,87
9	[SEP]	0,60
10	[SEP]	0,36
11	law	0,80
12	[SEP]	0,52

Figura 6

Mapas de calor de atención de la capa 5 de BERT



Tamaño de los modelos

La Tabla 8 presenta el número de parámetros de cada modelo (Figura 7).

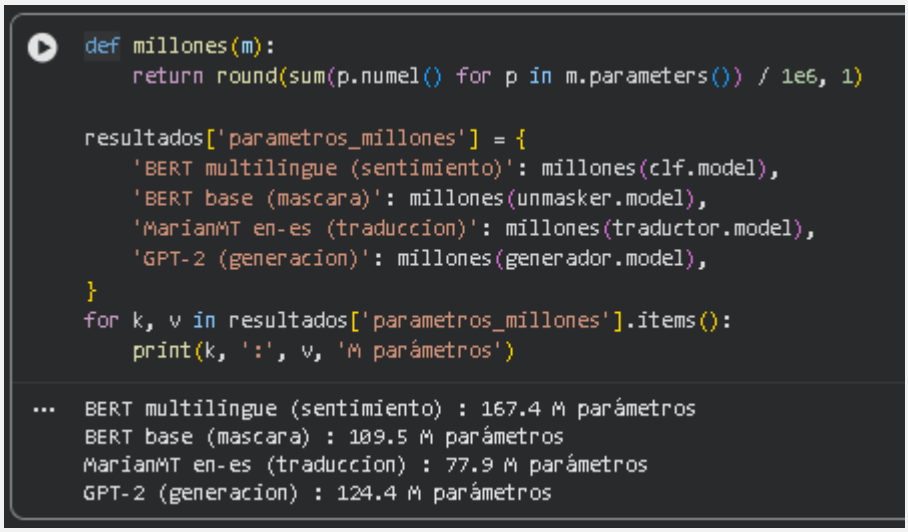
Tabla 8

Parámetros de cada modelo

Modelo	Parámetros (millones)
BERT multilingüe (sentimiento)	167,4
BERT base (máscara y atención)	109,5
Marian en-es (traducción)	77,9
GPT-2 (generación)	124,4

Figura 7

Salida de las celdas de atención por cabeza y de tamaño de los modelos



Análisis

Sentimiento (solo encoder). El modelo acertó la polaridad de los tres textos: dos con 5 estrellas y uno con 2 estrellas para el comentario negativo. La confianza fue mayor en inglés (0,84) que en español (0,51 y 0,48). Como el modelo reparte la probabilidad entre cinco clases, un valor de 0,48 está muy por encima del 0,20 que daría el azar, aunque indica menos seguridad que el 0,84 del texto en inglés. Este resultado coincide con lo descrito por Doshi (2020): un mismo bloque encoder, con una cabeza de clasificación al final, sirve para predecir etiquetas.

Relleno de máscara (solo encoder). BERT usa el contexto a ambos lados de la palabra oculta (Devlin et al., 2018). En "Paris is the capital of [MASK]" asignó 0,951 a "france", lo que muestra que la preentrenación le dio conocimiento factual. En la primera frase, en cambio, no propuso "attention" entre sus cinco opciones; eligió cantidades ("several", "two", "three") que son gramaticalmente correctas pero no corresponden a la expresión del artículo. El modelo predice lo plausible según el contexto de la frase, no lo que dice un texto concreto.

Traducción (encoder-decoder). Las tres traducciones son correctas y fluidas. En la segunda, el modelo tradujo "Transformer" como "transformador", un sustantivo común, y no como el nombre de la arquitectura. En la tercera concordó "its application" con "su aplicación" y "just" con "justa", lo cual es coherente con una atención cruzada que relaciona cada palabra generada con la entrada (Vaswani et al., 2017). La práctica no midió calidad con métricas como BLEU, que el artículo usa para comparar modelos.

Generación (solo decoder). GPT-2 produjo texto gramatical, pero con dos problemas: repitió la idea "predict the future" y afirmó algo falso sobre los Transformers. Esto es esperable en el modelo de menor tamaño de la familia (124,4 millones de parámetros) con decodificación voraz, que elige siempre la palabra más probable. El texto se corta tras 30 tokens, lo que refleja la generación palabra por palabra que describe Doshi (2020), donde cada palabra nueva se añade a la secuencia de entrada del decoder.

Atención. En la capa 5, tres cabezas relacionan "its" con palabras de la frase que tienen sentido gramatical: las cabezas 1 y 6 atienden a "application", y la cabeza 11 atiende a "law" con un peso de 0,80. Esta última coincide con la idea de resolución de anáforas que muestran Vaswani et al. (2017) en su Figura 4, donde "its" apunta a "Law". Sin embargo, siete de las doce cabezas (3, 4, 7, 8, 9, 10 y 12) tienen su máximo en el token [SEP], y las cabezas 2 y 5 apuntan a

"just" y "will" con pesos bajos (0,20 y 0,19). Es decir, no todas las cabezas codifican una relación lingüística clara, lo que respalda la observación del artículo de que las cabezas aprenden tareas distintas.

Tamaño. Los modelos usados tienen entre 77,9 y 167,4 millones de parámetros. Como referencia, el Transformer base del artículo tiene 65 millones y el grande, 213 millones (Vaswani et al., 2017, Tabla 3). El modelo Marian de la práctica queda entre ambos y es del mismo orden que el modelo base.

Limitaciones. La práctica usó pocos ejemplos, una sola ejecución y CPU, por lo que los tiempos (3,16 s y 8,82 s) no son comparables con los de GPU del artículo. El análisis de atención corresponde a una frase, una capa y el peso máximo por cabeza. Además, BERT base tiene 12 capas y 12 cabezas, frente a 6 capas y 8 cabezas del Transformer original, así que la comparación con las figuras del artículo es cualitativa.

Conclusiones

La lectura del artículo de Vaswani et al. (2017) y de Doshi (2020), junto con la práctica, permite concluir lo siguiente:

- **La atención sustituye a la recurrencia.** El Transformer relaciona cualquier par de posiciones con un número constante de operaciones y procesa la secuencia en paralelo, lo que explica su rapidez de entrenamiento frente a RNN y CNN.
- **La arquitectura es modular.** Los tres modelos de la práctica (BERT, Marian y GPT-2) usan piezas del mismo diseño: solo encoder para comprender texto, encoder-decoder para traducir y solo decoder para generar.
- **Los resultados fueron coherentes con la teoría.** Los modelos clasificaron, completaron y tradujeron correctamente los ejemplos, y GPT-2 mostró las

limitaciones esperables de un modelo pequeño con decodificación voraz: repetición e imprecisión factual.

- **La atención es observable, pero no siempre interpretable.** La cabeza 11 de la capa 5 asoció "its" con "law", como en la Figura 4 del artículo; sin embargo, siete de doce cabezas se concentraron en [SEP], por lo que los pesos de atención deben leerse con cautela.
- **La práctica tiene alcance limitado.** Se probaron pocos ejemplos, sin métricas cuantitativas y solo en CPU. Un trabajo posterior podría evaluar con BLEU o exactitud sobre un conjunto de datos y comparar varias capas y frases.

En conjunto, la práctica permitió reconocer la estructura de los Transformers, identificar el papel del mecanismo de atención y ver su aplicación en tareas reales de procesamiento de lenguaje natural.

Declaración sobre el uso de inteligencia artificial

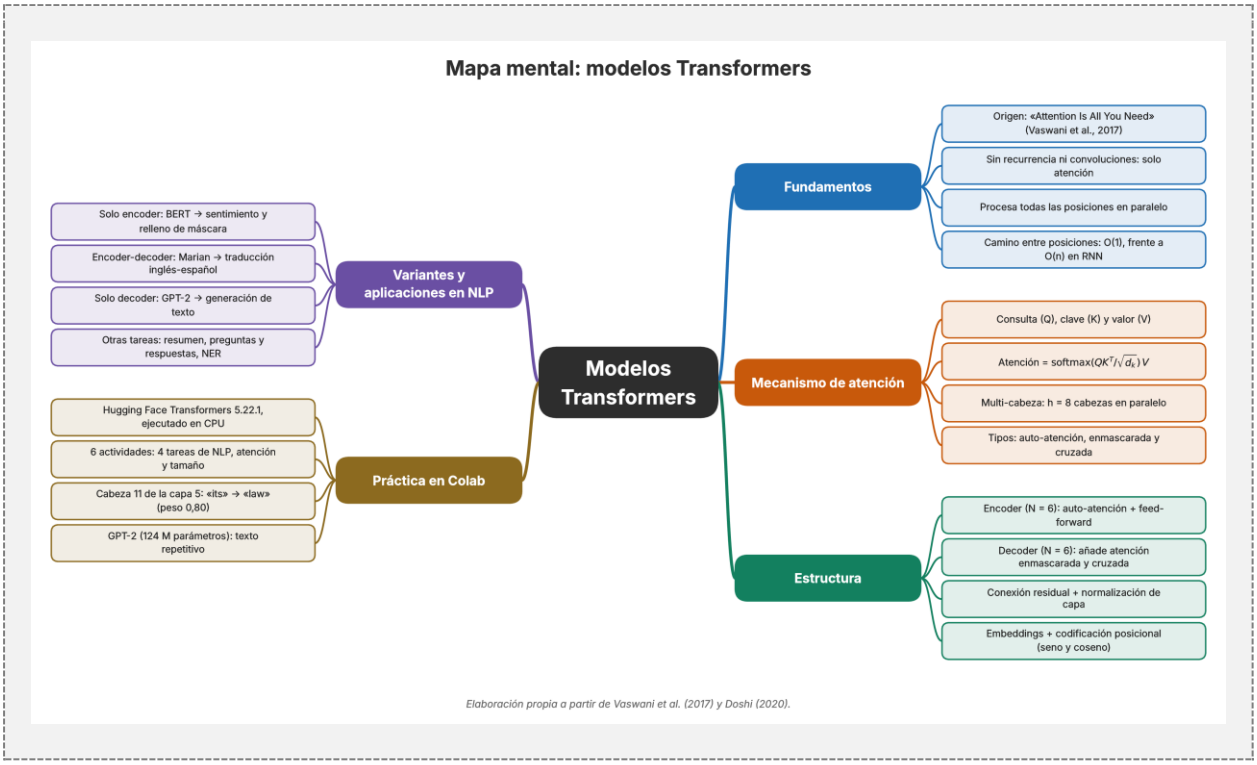
La práctica con modelos Transformers fue realizada con Claude (Sonnet 5.5), un asistente de inteligencia artificial de Anthropic (2026): Claude diseñó y programó el cuaderno de Google Colab, que el estudiante ejecutó para obtener los resultados presentados. El informe escrito, la selección de las fuentes, la interpretación de los resultados y las conclusiones son responsabilidad del estudiante.

Mapa mental

El mapa mental organiza los conceptos principales de los Transformers en cinco ramas: fundamentos, mecanismo de atención, estructura, variantes y aplicaciones, y práctica realizada (Figura 8). Se entrega también como archivo aparte.

Figura 8

Mapa mental de los modelos Transformers



Nota. Elaboración a partir de Vaswani et al. (2017) y Doshi (2020).

Repositorio github

Esta práctica se encuentra subida en el siguiente repositorio en github junto con el informe: https://github.com/NicolasRomero22/TallerTransformer

Referencias

Anthropic. (2026). *Claude* (Sonnet 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of deep bidirectional transformers for language understanding*. arXiv. https://arxiv.org/abs/1810.04805

Doshi, K. (2020, 13 de diciembre). *Transformers explained visually (Part 1): Overview of functionality*. Towards Data Science. https://towardsdatascience.com/transformers-explained-visually-part-1-overview-of-functionality-95a6dd460452/

Hugging Face. (s. f.). Pipeline. En *Transformers documentation*. https://huggingface.co/docs/transformers/pipeline_tutorial

Junczys-Dowmunt, M., Grundkiewicz, R., Dwojak, T., Hoang, H., Heafield, K., Neckermann, T., Seide, F., Germann, U., Aji, A. F., Bogoychev, N., Martins, A. F. T., & Birch, A. (2018). *Marian: Fast neural machine translation in C++*. arXiv. https://arxiv.org/abs/1804.00344

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*. https://arxiv.org/abs/1706.03762